

Python Decision Structures

part 2

- Aka “Error Handling”

CMSC 104 Section 2
October 5, 2026

Administrative notes

I'll return Quiz 1 today

Project 2 will be assigned today

- The due date is Monday, October 19, before midnight

Classwork 4 is due before midnight on Wednesday, October 7

Remember that UMBC is closed for “Fall Recess” next Monday and Tuesday - October 12 & 13.

Error handling

We've talked a number of times about errors that can crash your program

- Entering a name where a number is needed
- Trying to calculate the square root of a negative number
- Entering a negative number when you're trying to calculate a Fibonacci sequence
- Etc.

In general, a program crashing is considered poor form

- It's a sign of not-fully-formed coding skills

Error handling on your own

Suppose we ask the user to enter an integer

If the user enters the wrong characters, the program can crash

```
age = int(input("please enter your age in years"))  
  
if age >= 18:  
    print("Please vote; it's so important")  
else:  
    print("You can still get involved")
```

If the user enters “nineteen”
here your program crashes

Python provides a built-in method, “isdigit()” that looks at a string. If every character in that string is a digit, isdigit() returns True and so you can safely convert the string into an integer.

So the code on the previous page becomes:

```
age = input("Enter your age in years: ")
if age.isdigit():
    age = int(age)
    if age >= 18:
        print("Please vote; it's so important")
    else:
        print("You can still get involved")
else:
    print("I'm sorry, you have to enter a valid age using numbers")
```

But you don't have to do all the error
checking yourself

“try...except”

Python has a built in way of handling a set of common errors

Your program doesn't crash, it just lets the user know that something went wrong

```
try:
    x = int(input("Please enter a number: "))
    break
except ValueError:
    print("Oops! That was no valid number. Try again...")
```

If the user enters something that cannot be converted to an integer, Python throws an error of type “ValueError”

Syntax and semantics

Syntax:

```
try:
    #code that you want to run, that may crash
except <type of error>:
    #code to run when that error is detected by Python
```

Semantics:

It's pretty much like an "if" statement

- If your code works and no error condition is detected, the "try" section gets executed and the error handling is ignored
- If your code fails - like an "else" statement - anything in the "try" section after the error is not executed and the "except" clause - the error handling code - is executed
 - Anything in the "try" section prior to the error gets executed and not rolled back

Multiple error codes:

What if your code may cause one or more types of errors?

- Just like an “if..elif..else” statement, you can have multiple exceptions in your code

```
try:
    x = int(input("Please enter a number: "))
    y = "UMBC"
    print(x + y + x)

except ValueError:
    print("Oops! That was no valid number. Try again...")
except TypeError:
    print("Oops! That was a list; not a valid number. Try again...")
```

Where are these error codes defined?

See the official Python documentation at:

<https://docs.python.org/3/library/exceptions.html>

Common types:

exception **TypeError**

Raised when an operation or function is applied to an object of inappropriate type. The associated value is a string giving details about the type mismatch. This exception may be raised by user code to indicate that an attempted operation on an object is not supported, and is not meant to be. If an object is meant to support a given operation but has not yet provided an implementation, [NotImplementedError](#) is the proper exception to raise. Passing arguments of the wrong type (e.g. passing a [list](#) when an [int](#) is expected) should result in a [TypeError](#), but passing arguments with the wrong value (e.g. a number outside expected boundaries) should result in a [ValueError](#).

More error types

exception **ValueError**

Raised when an operation or function receives an argument that has the right type but an inappropriate value, and the situation is not described by a more precise exception such as [IndexError](#).

exception **ZeroDivisionError**

Raised when the second argument of a division or modulo operation is zero. The associated value is a string indicating the type of the operands and the operation.